DATA ENGINEERING & CLOUD · LESSON 7 OF 15

dbt: Models, Tests, Documentation and Lineage

Build transformations as tested, documented SQL models with dbt. Write models that reference each other with ref(), add schema and custom tests that run on every build, generate documentation and lineage, and organise a project so that changes are reviewed and deployed safely.

Gap-fill lesson | ✦ 2026 | Intermediate | ⏱ 120 min

By the end of this lesson you can

- Write a dbt model and reference other models with ref()
- Add generic tests (unique, not_null, accepted_values, relationships) and a custom test
- Generate documentation and read the lineage graph
- Organise a dbt project into staging, intermediate and marts layers

Datasets used

`superstore_sales.csv`

What dbt does

dbt (data build tool) turns SQL SELECT statements into managed tables and views, and adds the engineering practices software teams take for granted: dependencies, tests, documentation, version control and reviews. You write the transformation as SQL; dbt runs it in the right order, in your warehouse, and checks it.

CASE STUDY **A metric that changed without notice**

An analyst changed how "active customer" is calculated inside a dashboard query. Three teams reported different numbers for a month, and nobody knew which was right. In dbt, the definition lives in one model, is tested and documented, and every downstream model references it, so a change is visible in review and in the lineage graph.

Models and ref()

A **model** is a SQL file that defines one table or view. The `ref()` function points to another model; dbt uses these references to build the dependency graph, so you never hard-code the order.

SQL

```sql
-- models/staging/stg_sales.sql
select
    row_id                      as sale_id,
    order_id,
    cast(order_date as date)    as order_date,
    customer_id,
    cast(sales as numeric(12, 2))  as sales,
    cast(profit as numeric(12, 2)) as profit
from {{ source('raw', 'superstore_sales') }}
```

SQL

```sql
-- models/marts/fct_monthly_sales.sql
select
    date_trunc('month', s.order_date) as month,
    c.region,
    sum(s.sales)  as sales,
    sum(s.profit) as profit
from {{ ref('stg_sales') }} as s
join {{ ref('stg_customers') }} as c using (customer_id)
group by 1, 2
```

`source()` names raw tables that dbt does not build. `ref()` names models that dbt builds. Both create the lineage links you will see in the documentation.

Layers in a dbt project

Layer	Folder	Rule
Staging	`models/staging`	One model per source table; renaming, typing and light cleaning only
Intermediate	`models/intermediate`	Reusable joins and logic that several marts need
Marts	`models/marts`	Business tables for reporting, with tests and descriptions

Keep business logic in marts and intermediate models, not in staging. A staging model that contains a revenue definition gets copied into every report that uses it.

Tests: generic and custom

Tests are queries that return the rows that break a rule. dbt runs them after each build; a failing test fails the build.

YAML

```yaml
# models/marts/schema.yml
version: 2
models:
  - name: fct_monthly_sales
    description: "Sales and profit per month and region. One row per month and region."
    columns:
      - name: month
        tests: [not_null]
      - name: region
        tests:
          - not_null
          - accepted_values:
              values: ["North", "South", "East", "West"]
      - name: sales
        tests: [not_null]
```

Generic tests include `unique`, `not_null`, `accepted_values` and `relationships` (a foreign key exists in the parent table). A **custom test** is a SQL query that returns the failing rows, for a business rule:

SQL

```sql
-- tests/assert_sales_not_negative.sql
-- The test passes when this query returns no rows
select sale_id, sales
from {{ ref('stg_sales') }}
where sales < 0
```

::::tip Test the grain, not only the columns The most useful test on a fact table checks its grain: that one combination of keys appears once. Add a `unique` test on a surrogate key, or a test on the combination of key columns with `dbt_utils.unique_combination_of_columns` if you use that package. ::::

Documentation and lineage

Descriptions in the YAML become documentation. Running `dbt docs generate` and `dbt docs serve` produces a website with every model, column description, test and the **lineage graph**: which

sources and models feed each model, and which dashboards depend on it. Lineage answers "if I change this column, what breaks?" before the change is made.

Running and reviewing changes

Terminal

```
dbt deps          # install packages
dbt build         # run the models and their tests in dependency order
dbt build --select fct_monthly_sales+   # the model and everything downstream of it
dbt test          # run tests only
```

A safe change follows the same path as software: a branch, a pull request that shows the changed SQL and the test results, a review, and a deployment that runs `dbt build` in a staging environment before production. Changes that alter a metric should include a comparison of old and new values for the last few months.

::::warn Incremental models need care Incremental models process only new rows for speed. A change to the logic does not rewrite old rows unless you force a full refresh. After changing an incremental model's logic, run a full refresh in staging and compare the totals with the previous version. ::::

Documentation habits that pay off

- Write a one-sentence description for every mart and every key column.
- Name the grain in the description ("one row per order line").
- Record the owner of each model and the refresh schedule.
- Keep the definition of each business metric in one model, and link to it from dashboards.

INTERVIEW CORNER **What does dbt add to plain SQL scripts?**

Dependencies between models through ref(), a single build command that runs them in order, tests that run on every build, generated documentation and lineage, and a project structure that works with version control and code review. The SQL is the same; the discipline around it is what makes it maintainable.

::::key What to remember - A dbt model is a SELECT; ref() creates dependencies and lineage. - Use staging for cleaning, intermediate for reuse, and marts for business tables. - Tests return failing rows; a failing test fails the build. - Test the grain of fact tables, and write custom tests for business rules. - Review metric changes with before-and-after comparisons. ::::

Quick quiz

5 questions · tap an answer to check it

1. What does `ref('stg_sales')` do?

- **A** It copies the table into the current file
- **B** It references another model and tells dbt to build it first
- **C** It deletes the staging model
- **D** It creates a test

Answer: B. ref() creates the dependency graph.

2. What does a dbt test return when it passes?

- **A** A row count of one thousand
- **B** No rows, because the query finds no failing cases
- **C** A chart
- **D** An error

Answer: B. A test is a query for violations; zero rows means pass.

3. Which layer should hold a revenue definition used by several reports?

- **A** Raw sources
- **B** Staging
- **C** Marts or intermediate models
- **D** Dashboards only

Answer: C. Business logic belongs in shared models, not in copies.

4. What question does the lineage graph answer?

A Who is the best analyst

B If I change this model, which models and dashboards are affected?

C How long the query takes

D Which colour to use

Answer: B. Lineage shows dependencies before a change.

5. After changing an incremental model's logic, what should you do?

A Nothing; old rows update automatically

B Run a full refresh in staging and compare totals with the previous version

C Delete the model

D Skip testing

Answer: B. Incremental models keep old rows unless forced to rebuild.

EXERCISE **Add a test and a description**

Take the `fct_monthly_sales` example and add (a) a `unique` test on the combination of month and region (or a surrogate key), (b) a custom test that fails when a month has negative total sales, and (c) a description of the grain. Explain in three sentences what a failing test would mean for the dashboard.

Show solution (worked answer)

Expected: the combination test checks grain; the custom test returns rows where monthly sales are negative; a failure stops the build so the dashboard is not refreshed with the bad numbers.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.